

Introduction to class constructor



Example

```
/* Traditional "Hello World!" program. */  
  
// package pack1;  
// import java.lang.System;  
class prog1  
{  
    public static void main (String args[])  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Main method

```
public static void main (String args[])  
{  
    //instructions  
}
```

- The **main** method must always be defined as **public**: to make it publicly accessible,
- **static**: to declare it as a class member and
- **void**: returns no value
- **args[]**: parameter is an array of class String. It provides access to command line parameters

System class

```
System.out.println("Hello World!");
```

- invokes `println` method on object named `out` (variable of type `java.io.PrintStream`), which is a member of `System` class.
- The `println` method takes a `String` parameter and displays it on the console

Class definition

```
class C1
```

```
{
```

```
    int i;
```

```
    char ch;
```

```
    public void name()
```

```
    {...}
```

```
    int get(int b)
```

```
    {...}
```

```
}
```

Access members of a class

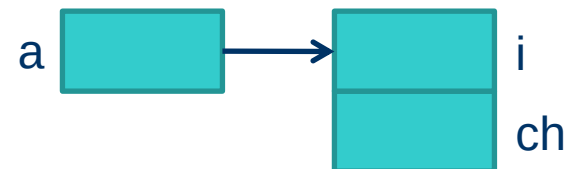
Class C1

```
{  
    int i;  
    char ch;  
    void set()  
        { i=20; }  
    int get()  
        { return i; }  
}
```

How to create two objects of same class
with different data elements?

How to access member of a class ?

```
C1 a= new C1();  
a.i;  
a.ch;  
a.set();
```



Constructor

- Similar as a method in a class
- Same name as class name
- No return type as methods

Why we need constructors?

- Initialize the members

Default cons (if we not defined)

- No parameter

- Ex: `C1() {
 }`

Parameterized constructor

```
C1(int n)
```

```
{
```

```
    i=n;
```

```
}
```

```
C1(int n, char c)
```

```
{
```

```
    i=n;
```

```
    ch=c;
```

```
}
```

- Called when object initialized
- Can define any number of constructors

Example 2: two classes

```
class C1
{
    int i;
    char ch;
    void set()
        { i=20;}

    int get()
        {return i;}
}
```

```
public class C2
{
    public static void main(String args[])
    {
        C1 obj= new C1();
        int b;
        obj.set();
        b= obj.get();
        System.out.println("i="+ obj.i);
        System.out.println("i="+ b);
    }
}
```

Example 3: two classes uses cons.

```
class C1
{
    int i;
    char ch;
    void set()
        { i=20;}
    int get()
        {return i;}
}
```

```
C1 (int in, char c)
{ i=in; ch=c;
}
}
```

```
public class C2
{
    public static void main(String args[])
    {
        C1 obj= new C1(20,'g');
        System.out.println("i="+ obj.i);
        System.out.println("i="+ obj.ch);
    }
}
```

Example 4: single class

```
public class C1
{
    int i;
    char ch;
    void set()
        { i=20;}

    int get()
        {return i;}
```

```
public static void main(String args[])
{
    C1 a= new C1();
    int b;
    a.set();
    b=a.get();
    System.out.println("i="+ a.i);
    System.out.println("i="+ b);
}
}
```

Applet Example 1

```
/*  
<APPLET CODE="app1.class" WIDTH=150 HEIGHT=100>  
</APPLET>  
*/  
  
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class app1 extends Applet {  
    public void paint (Graphics g) {  
        g.drawString("Hello!",50,20);  
    }  
}
```

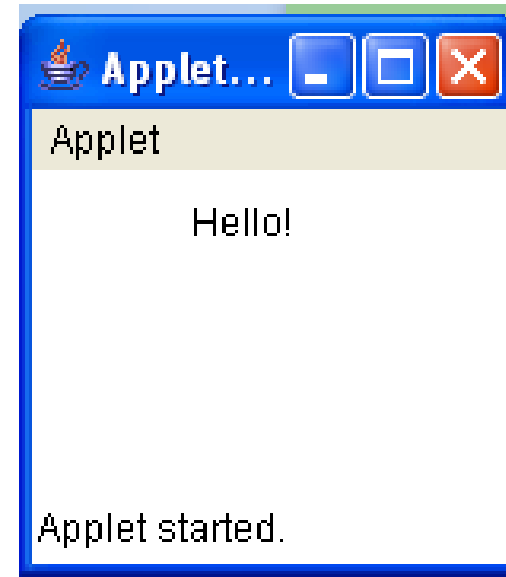
Applet program execution

Compile

```
javac app1.java
```

Execution

```
appletviewer app1.java
```



Execution through HTML file

```
<HTML>
  <HEAD>
    <TITLE> A simple Program</TITLE>
  </HEAD>
  <BODY> Here is the output:
    <APPLET CODE="app1.class" WIDTH=150 HEIGHT=100>
    </APPLET>
  <BODY>
</HTML>
```

Store with name app1.htm

Execute from browser: C:\java\app1.htm